

VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS

FAKULTETAS

Programų sistemų kūrimo

Komandinio darbo ataskaita

Darbą atliko:
Edvinas Burba, Augustas Budnikas, Leonardas Sinkevičius, Gustas Šukys, Adomas
Šileikis

Komandos tikslai

Bendri komandos tikslai:

- Sukurti bilietų pardavimo ir renginių valdymo sistemą su mokėjimų integracija, QR kodais ir el. laiškais.
- Įdiegti sistemą į produkcinę debesų aplinką (Vercel front-end'ui, Google Cloud Run API, Google Cloud SQL duomenų bazei) per CI/CD pipeline ir užtikrinti jos prieinamumą internete.
- Įsisavinti modernių technologijų rinkinį: .NET, Next.js 16, React 19, PostgreSQL, Docker, GitHub Actions.
- Išmokti efektyviai dirbti agile komandoje su aiškiomis rolėmis, sprintais ir kodo peržiūromis.

Asmeniniai tikslai:

- Adomas (Tech Lead) - gilinti žinias programų sistemų architektūroje, išmokti efektyviai atlikti kodo peržiūras, priimti techninius sprendimus dėl bibliotekų pasirinkimo bei sluoksnių išdėstymo.
- Gustas (Projekto vadovas) - išmokti planuoti sprintus, koordinuoti tarp-komandinį darbą ir vesti sprint apžvalgas. Tobulinti komunikacijos su komandos nariais.
- Augustas (Back-end) - gilinti žinias .NET, PostgreSQL ir Entity Framework Core, įsisavinti Stripe mokėjimų integraciją bei webhook valdymą, taikyti Clean Architecture principus realiaame projekte.
- Leonardas (Front-end) - gilinti žinias Next.js 16 (App Router, SSR), React 19, TailwindCSS. Išmokti kurti interaktyvius komponentus (pvz., vietų žemėlapis) ir vartotojo dashboard'us.
- Edvinas (Dizaineris) - tobulinti UI/UX dizaino įgūdžius su Figma, kurti wireframe'us bei aukšto detalumo prototipus, suformuoti dizaino sistemą (spalvos, tipografija, komponentai) ir bendradarbiauti su front-end komanda.

Komandinės veiklos taisyklės:

- Visas kodas eina per Pull Request'us, kuriuos peržiūri Tech Lead (Adomas) prieš sujungiant į master branch.
- Naudojami aiškūs commit'ų pavadinimai pagal Conventional Commits standartą (feat, fix su scope API / UI / CICD).
- Projekto vadovas (Gustas) planuoja sprintus, paskirsto užduotis, veda stand-up'us.
- Dizaineris (Edvinas) paruošia wireframe'us bent vieną sprintą į priekį, kad front-end komanda turėtų aiškų pavyzdį dar prieš darbo pradžią.
- Lygiagretus darbas: back-end ir front-end komandos dirba tuo pačiu metu, sinchronizuojantis per OpenAPI/Swagger kontraktą ir auto-generuojamą TypeScript SDK.
- Bet kokia abejonė ar nesklandumas keliami iškart komandos kanale, o ne palaikomi sau - tai apsaugo nuo dvigubo darbo.

Rolės ir atsakomybės:

- Adomas - Tech Lead. Architektūros sprendimai, kodo peržiūros (PR review), bibliotekų pasirinkimas, sluoksnių išdėstymas, kodo kokybės standartų užtikrinimas.
- Gustas - Projekto vadovas (PM). Sprintų planavimas, backlog'o priežiūra, terminų stebėjimas, kliūčių pašalinimas.
- Augustas - Back-end inžinierius. .NET API kūrimas, PostgreSQL duomenų bazės modeliavimas, migracijos, autentifikacija, Stripe integracija, el. laiškų sistema, saugumas.
- Leonardas - Front-end inžinierius. Next.js puslapiai, React komponentai, vietų žemėlapis UI, admin dashboard'as, integracija su API per auto-generuojamą SDK.
- Edvinas - Dizaineris. Wireframe'ai (Figma), aukšto detalumo prototipai, dizaino sistema, UX peržiūra, naudotojų scenarijų vizualizavimas.

Projekto tikslai

Projekto aprašymas

Esamos bilietų platformos Lietuvoje pasižymi pasenusia naudotojo patirtimi ir aukštais komisiniais. Mažesni renginių organizatoriai neturi patogaus įrankio savo renginiams valdyti.

Projekto tikslas - sukurti modernią bilietų pardavimo sistemą, kuri tiks trims naudotojų grupėms: pirkėjams, organizatoriams ir skeneriams (darbuotojams prie įėjimo).
Produktas - web aplikacija su .NET back-end, Next.js front-end ir PostgreSQL duomenų baze. Sistema sukurta 2026 m. kovo - gegužės mėn. (apie 10 savaičių).

Projekto apimtis

Sistema apima:

- Naudotojus: pirkėjus, administratorius, skenerius.
- Procesus: registraciją, renginių valdymą, bilietų pirkimą, patikrą, atsiliepimus, nuolaidas, analitiką.
- Technologijas: .NET, Hangfire, Next.js, React, PostgreSQL, Docker, Stripe, Vercel, Google Cloud Run, Google Cloud SQL.

Neįtraukta: mobilioji aplikacija, fizinių bilietų spausdinimas.

Projekto pateiktys

Pagrindinės projekto pateiktys:

Nr.	Pateiktis	Aprašymas
1	Naudotojo autentifikacija	Registracija, prisijungimas su JWT, naudotojo profilis.
2	Renginių katalogas	Renginių sąrašas su paieška ir individualūs puslapiai.

3	Vietų pasirinkimas	Interaktyvus vietų žemėlapis perkant bilietą.
4	Mokėjimai (Stripe)	Pilnas pirkimo srautas su Stripe ir webhook patvirtinimu.
5	QR kodų bilietai	Po apmokėjimo generuojamas QR kodas, prieinamas profilyje.
6	Administravimas	Dashboard'as renginiams, bilietų tipams ir vietoms valdyti.
7	Bilietų patikra (check-in)	QR kodo skenavimas prie įėjimo.
8	Skenerių valdymas	Atskira rolė ir sąsaja darbuotojams prie įėjimo.
9	El. laiškai (Hangfire)	Patvirtinimai ir priminimai per Hangfire delayed execution.
10	Nuolaidų kodai	Kodų sukūrimas ir taikymas perkant.
11	Atsiliepimai	Naudotojai gali palikti atsiliepimą po renginio.

12	Analitika	Statistikos puslapis su grafikais.
13	CI/CD	Automatinis deployment'as: Vercel (UI) + Google Cloud Run (API) per GitHub Actions.

Naudoti metodai

Back-end: C# / .NET su Clean Architecture, PostgreSQL, Entity Framework Core, Hangfire užduočių planuotojui (delayed execution - el. laiškų ir priminimų siuntimas).

Front-end: TypeScript, Next.js 16, React 19, TailwindCSS, shadcn/ui.

Dizainas: Figma - wireframe'ai, prototipai, dizaino sistema, komponentų biblioteka.

Integracijos: Stripe (mokėjimai), SMTP (el. laiškai).

DevOps: Docker, GitHub Actions, Docker Hub. Hostingas: Vercel (UI), Google Cloud Run (API), Google Cloud SQL (duomenų bazė).

Projekto valdymas: Jira backlog'ui, kasdieniai stand-up'ai, sprintų retrospektyvos.

Bendradarbiavimas: Git, Pull Request'ai su Tech Lead'o peržiūra, OpenAPI/Swagger tarp back-end ir front-end, bendri socialiniai kanalai greitiems klausimams.

Darbo eiga

Projektas vyko 10 savaitių (2026-03-10 - 2026-05-22) penkiais dviejų savaitių sprintais. Gustas (PM) suplanuodavo sprinto apimtį, paskirstydavo užduotis, vedavo apžvalgas. Adomas (Tech Lead) peržiūrėdavo visus Pull Request'us ir priimdavo techninius sprendimus.

1 sprintas (03-10 - 03-23): projekto karkasas ir pagrindai. Edvinas paruošė dizaino sistemą Figma, bei pirmus wireframe'us pagrindiniams puslapiams. Leonardas sukūrė pradinę Next.js struktūrą, autentifikacijos UI ir renginių sąrašo puslapį. Augustas paruošė .NET API skeletą su Clean Architecture, EF Core migracijas ir JWT autentifikaciją. Adomas suderino bendrą architektūrą ir technologijų rinkinį. Gustas paruošė pradinį backlog'ą Jira.

2 sprintas (03-24 - 04-06): bilietų pirkimo srautas. Edvinas paruošė pirkimo srauto wireframe'us (renginio puslapis → bilieto pasirinkimas → krepšelis → mokėjimas). Augustas integravo Stripe mokėjimus su webhook'ais. Leonardas įgyvendino renginio puslapį ir bilieto pasirinkimo UI.

3 sprintas (04-07 - 04-20): naudotojo profilis, QR kodai, admin pradžia. Edvinas paruošė admin dashboard'o pirmus mokap'us ir naudotojo profilio dizainą. Augustas pridėjo bilietų išdavimą, QR kodų generavimą ir užsakymų API. Leonardas sukūrė naudotojo profilio puslapį su bilietais ir admin puslapio pirmąją versiją. Adomas atliko architektūros peržiūrą prieš pradėdant admin funkcionalumus.

4 sprintas (04-21 - 05-04): pilnas administravimas ir CI/CD. Edvinas paruošė admin dashboard mockup'us renginiams, bilietų tipams ir naudotojų valdymui. Augustas užbaigė admin API. Leonardas įgyvendino pilną admin dashboard'ą. Gustas su Adomu kartu nustatė CI/CD pipeline - Docker Hub publikavimą, Google Cloud Run deployment'ą per GitHub Actions. Sprinto pabaigoje pristatyta pirmoji veikianti produkcijos versija.

5 sprintas (05-05 - 05-22): pažangūs funkcionalumai ir baigimas. Edvinas paruošė interaktyvaus vietų žemėlapiu dizainą, atsiliepimų ir skenerių UI mokap'us. Augustas įgyvendino vietų žemėlapi, el. laiškų siuntimą per Hangfire (delayed execution priminimams prieš renginį), nuolaidų kodus, atsiliepimus, analitiką ir skenerių API. Leonardas įgyvendino atitinkamus UI komponentus, įskaitant interaktyvų vietų žemėlapi, analitikos dashboard'ą su grafikais ir skenerių sąsają. Augustas atliko saugumo pataisymus ir reikalavimų suderinimą. Komanda atliko galutinę retrospektyvą ir Gustas paruošė pateikimo reikalavimus.

Bendradarbiavimas:

Komanda dirbo lygiagrečiai pagal aiškiai apibrėžtas roles. Edvinas (Dizaineris) pradėjo darbą anksčiausiai - jo wireframe'ai buvo pateikiami bent vieną sprintą į priekį, todėl Leonardas (Front-end) galėjo iš karto pradėti darbą. Augustas (Back-end) ir Leonardas dirbo lygiagrečiai, sinchronizuodamiesi per OpenAPI/Swagger kontraktą bei automatiškai generuojamą TypeScript SDK - tai eliminavo tipų neatitikimus tarp sluoksnių.

Adomas (Tech Lead) prisiimdavo visų Pull Request'ų peržiūrą - tikrindavo kodo kokybę, architektūros principus, saugumą. Sudėtingesniems sprendimams (pvz., autentifikacijos schemas pasirinkimas, vietų žemėlapiu duomenų modelis) jis suorganizuodavo trumpą techninę diskusiją su atitinkamu komandos nariu.

Gustas (PM) vedavo trumpus „stand-up“ susitikimus, dėl kliūčių perkeldavo užduotis arba paprašydavo kito komandos nario pagalbos. Sprintų pradžioje Gustas paruošdavo aiškų backlog'ą, o pabaigoje vedavo apžvalgą su klausimais „kas pavyko gerai“, „kas nepavyko“, „ką keisti kitame sprints“.

Komandos bendri socialiniai kanalai buvo naudojami neformaliai bendravimui, nuorodų dalinimuisi ir greitiems klausimams. Visi formalūs sprendimai ir architektūros aprašymai būdavo užfiksuoti Pull Request aprašuose, kad išliktų istorija.

Iššūkiai ir jų sprendimas:

1. Hosting platformos migracija (Railway → Google Cloud Run). Pradžioje back-end API buvo dedamas į Railway, tačiau platformos free tier neužteko stabiliam darbui - atsirado nuolatinių apribojimų ir resursų nepriteklius. Sprendimas: 4 sprints Adomas (Tech Lead) nusprendė migruoti API į Google Cloud Run su Google Cloud SQL duomenų baze. Migracija pareikalavo perrašyti Dockerfile, atnaujinti GitHub Actions workflow ir perkonfigūruoti aplinkos kintamuosius, bet rezultatas - stabili produkcijos aplinka be free tier apribojimų.

2. Tipų neatitikimai tarp back-end ir front-end. Pradžioje front-end gaudavo neteisingus duomenų tipus iš API, todėl tekdavo derintis rankomis arba perdaryti komponentus. Sprendimas: 2 sprinte Adomas pasiūlė įdiegti @hey-api/openapi-ts automatinį SDK generatorių, kuris pagal Swagger spec sukuria TypeScript tipus - tai pašalino tipo neatitikimus.

3. Dizaino vėlavimas. Pirmajame sprinte Edvinas dar nebuvo paruošęs visų wireframe'ų, todėl Leonardas turėjo improvizuoti ir vėliau perdaryti kelis komponentus. Sprendimas: Gustas pakeitė sprintų planavimą taip, kad Edvinas visada dirbtų vienu sprintu į priekį - iki kito sprinto pradžios visi reikalingi dizainai jau būna paruošti.

4. Stripe webhook'ų testavimas vietoje. Augustui buvo sunku išbandyti Stripe webhook'us lokaliai - be viešo URL Stripe negali siųsti įvykių. Sprendimas: Adomas pasiūlė naudoti Stripe CLI komandą „stripe listen --forward-to“, kuri sukuria tunelį į localhost.

5. Pull Request'ų peržiūros vėlavimas. Pradžioje Adomas (vienintelis peržiūrintis) negalėjo aprėpti visų PR'ų per dieną - komandos nariai buvo blokuojami. Sprendimas: paruošė PR peržiūros checklist'ą, pagal kurį kiti komandos nariai galėjo daryti pirminę peržiūrą tarpusavyje.

Pasiekti rezultatai

Planuotų ir faktinių rezultatų palyginimas. Visi planuoti funkcionalumai įgyvendinti.

Nr.	Planuoti rezultatai	Faktiniai rezultatai
1	Naudotojo autentifikacija	Įgyvendinta su JWT ir rolėmis.
2	Renginių katalogas su paieška	Įgyvendinta.
3	Vietų pasirinkimas žemėlapyje	Įgyvendinta.
4	Stripe mokėjimai	Įgyvendinta.
5	QR kodų bilietai	Įgyvendinta.
6	Administravimo dashboard'as	Įgyvendinta.
7	Bilietai patikra (check-in)	Įgyvendinta.
8	Skenerių rolė ir sąsaja	Įgyvendinta.
9	Automatiniai el. laiškai (Hangfire)	Įgyvendinta su Hangfire delayed execution priminimams prieš renginį.
10	Nuolaidų kodai	Įgyvendinta.
11	Atsiliepimai	Įgyvendinta.
12	Analitika su grafikais	Įgyvendinta.
13	CI/CD pipeline	Įgyvendinta - Vercel (UI) + Google Cloud Run (API) + Google Cloud SQL (DB).

Išmoktos pamokos

Pamokos iš projekto eigos:

Patirtis	Pamoka
Hosting platformos pasirinkimas užtruko - pradžioje pasirinkta Railway back-end'ui, bet jos free tier neužteko stabiliam darbui, todėl 4 sprints teko migruoti į Google Cloud Run su Cloud SQL. Migracija pareikalavo nemažai laiko, kuris galėjo būti skirtas funkcionalumui kurti.	Hosting platformą reikia įvertinti ne tik pagal funkcionalumą, bet ir pagal free tier apribojimus dar prieš pradedant projektą. Geriau iš karto pasirinkti platformą, kuri tikrai pakels projekto poreikius (CPU, RAM, build time), nei keisti platformą vidury projekto. Tech Lead turėtų patvirtinti deployment'o strategiją per pirmąjį sprintą.
API ir UI komandos paraleliai dirbdamos kelis kartus susidūrė su duomenų tipų neatitikimais - kol nebuvo įdiegtas automatinis SDK generavimas iš OpenAPI specifikacijos, front-end gaudavo neteisingus tipus ir tekdavo perdaryti komponentus arba derintis ranka.	Verta nuo pirmos dienos turėti automatiškai generuojamą tipų kontraktą tarp back-end ir front-end. Tai eliminuoja tipų neatitikimus, leidžia abiems pusėms dirbti lygiagrečiai be konfliktų ir sumažina darbo perdarymo kiekį.
Kelias kartus front-end darbai prasidėjo be aiškaus dizaino - Leonardui tekdavo improvizuoti, o vėliau Edvinui paruošus tikslesnį wireframe'ą, komponentus reikėjo perdaryti. Tai sukėlė nereikalingo darbo ir nepasitenkinimo.	Dizaineris turi paruošti wireframe'us bent vieną sprintą į priekį, kad front-end komanda turėtų aiškų pavyzdį dar prieš darbo pradžią. Jei dizaino dar nėra - geriau pradėti nuo struktūrinių elementų (layout, navigacija), o detales palikti vėliau.
Kodo peržiūros (PR review) kartais užtrukdavo daugiau nei dieną, nes Tech Lead buvo vienas asmuo, atsakingas už visus PR'us. Tai blokavo komandos narius ir vertė kurti ilgas šakas.	Reikia nustatyti aiškų PR peržiūros laiką (pvz., 24 val. SLA) arba apmokyti dar vieną komandos narį atlikti peržiūras pagal aiškų checklist'ą. Trumpi, fokusuoti PR'ai (vienas funkcionalumas) peržiūrimi greičiau - darbą reikia skaidyti į mažesnius vienetus.